

Capstone Milestone 04

Merge #14 Integration & Runtime Hardening

Arad Fadaei, Mahboobeh Yasini, Johnpaul Tamburro

Version 4.0, March 27, 2026

Table of Contents

1. Milestone Goal	1
2. Merge #14 Scope Summary	2
3. Summary of Completed Work	3
3.1. Deployment and Environment Standardization	3
3.2. API Hardening and Contract Expansion	3
3.3. Programmatic Gameplay Logic Maturity	3
3.4. Generation and Narrative Reliability Improvements	4
3.5. Frontend Command and Visualization Expansion	4
3.6. Test Suite Expansion	5
4. Architecture Impact	6
4.1. Operational Readiness Increased	6
4.2. Reliability Shifted from Implicit to Explicit	6
4.3. Programmatic Core Extended Beyond Intent Parsing	6
5. Risks and Known Gaps	7
5.1. Strict LLM Mode Remains Credential/Quota Sensitive	7
5.2. Game Balance Requires Playtest Tuning	7

Chapter 1. Milestone Goal

The Goal: Consolidate the programmatic gameplay migration into a deployable, test-backed MVP baseline by hardening runtime behavior, expanding API/frontend contracts, and reducing failure modes in save/load, intent handling, and generation.

Reference Merge Analyzed: [f4c7ddd](#) (Merge pull request #14 from [Downmoto/chore/programmatic-game-logic-migration](#)).

Chapter 2. Merge #14 Scope Summary

This milestone report is based on the latest merged PR (#14), which introduced broad backend, frontend, test, and deployment updates.

- **Total change volume:** 28 files changed.
- **Key focus areas:** runtime mode reliability, API contract expansion, programmatic turn logic maturity, containerized launch path, and backend test coverage growth.

Chapter 3. Summary of Completed Work

3.1. Deployment and Environment Standardization

The project now supports a first-class container launch path for both frontend and backend.

Implemented artifacts:

- `Dockerfile.backend`
- `Dockerfile.frontend`
- `docker-compose.yml`
- `.env.example`
- `.dockerignore`

Operational outcomes:

- backend and frontend now expose explicit health checks
- environment-driven host/port/origin/provider configuration is standardized
- Ollama profile is available through Compose for local model workflows

3.2. API Hardening and Contract Expansion

Backend API behavior was made safer and more diagnostic-friendly.

Key additions and refinements:

- save/load path normalization with directory boundary checks (`data/` confinement)
- `POST /api/load-game` for restoring saved sessions
- `GET /api/game/map` for room-graph metadata
- expanded `POST /api/game/turn` response with additive state snapshot:
 - current room id
 - player hp/max hp
 - inventory size
 - history size
- health payload now includes endpoint visibility used by frontend diagnostics

Result: frontend command and diagnostics flows can operate against explicit, stable API contracts.

3.3. Programmatic Gameplay Logic Maturity

Core turn resolution now supports richer deterministic behavior while preserving LLM augmentation.

Notable gameplay enhancements:

- new deterministic **health** action
- combat now supports damage + retaliation + hp updates
- explicit game-over signaling (**game_over**) when player hp reaches zero
- visited-room return messaging distinction (**You return to ...**)
- query action flow for factual questions, with persisted facts on rooms/items/enemies (**extra_info**)
- partial enemy-name query matching improvements for more forgiving player input

Result: the game loop is less brittle and more stateful even when LLM behavior is limited or intermittent.

3.4. Generation and Narrative Reliability Improvements

Dungeon generation and narration received structural reliability upgrades.

Implemented mechanics:

- single-call full-dungeon JSON generation path with retries and schema checks
- strict-LLM safeguards in generation paths (fail loudly in strict mode, controlled fallback otherwise)
- expanded programmatic biome/content tables with progression-aware enemy hp scaling
- narrative context enrichment via recent-turn window + rolling summary controls
- canonical room footer enforcement (**Directions**, **Items**) to keep output aligned with game state

Result: world generation and turn narration are more coherent, resilient, and contract-consistent.

3.5. Frontend Command and Visualization Expansion

Terminal command UX now covers more lifecycle and debugging workflows.

New or updated frontend capabilities:

- **/load-game** command integration
- **/map** command integration with graph-based ASCII room layout rendering
- updated API client types for map payload and turn state snapshot
- command help text and startup messaging updated to match current backend feature set

Result: users can recover sessions and inspect dungeon topology directly from the terminal interface.

3.6. Test Suite Expansion

Backend testing was significantly expanded in merge #14.

New test modules:

- `test_api_map.py`
- `test_game_turns.py`
- `test_generator_programmatic.py`
- `test_intent.py`

Test inventory at merge tip:

- `test_api_map.py`: 5 tests
- `test_game_turns.py`: 18 tests
- `test_generator_programmatic.py`: 6 tests
- `test_intent.py`: 14 tests
- **Total backend tests in these modules: 43**

Coverage focus includes map API shape, save/load error handling, state snapshots, strict-LLM behavior, parser behavior, combat and hp transitions, query fact persistence, and generation safeguards.

Chapter 4. Architecture Impact

4.1. Operational Readiness Increased

Containerization and env templates convert the project from dev-only iteration into a repeatable launch workflow suitable for milestone demos.

4.2. Reliability Shifted from Implicit to Explicit

Path validation, strict-mode behavior, endpoint diagnostics, and typed state snapshots reduce hidden failure modes and improve troubleshooting speed.

4.3. Programmatic Core Extended Beyond Intent Parsing

The deterministic core now spans richer combat, factual query persistence, and map-state exposure, reducing dependence on best-case LLM behavior for baseline playability.

Chapter 5. Risks and Known Gaps

5.1. Strict LLM Mode Remains Credential/Quota Sensitive

`GAME_MODE=llm` intentionally fails hard when provider requirements are unmet. This is correct by design but can affect demo reliability without preflight checks.

5.2. Game Balance Requires Playtest Tuning

Combat and spawn scaling are functional, but pacing, damage curves, and difficulty progression still need balance passes.